

Experiment: XOR Neural Network using Backpropagation (2-2-1 Architecture)

Aim

To implement a feedforward neural network to solve the XOR problem and visualize loss, accuracy, and final decision boundary.

Theory

XOR is a nonlinear classification problem that cannot be solved using a single perceptron. A multilayer neural network with backpropagation is used. Activation function used is Sigmoid and loss function is Mean Squared Error.

Requirements

Python, NumPy, Matplotlib

Source Code

```
import numpy as np

import matplotlib.pyplot as plt

# ----- Activation Functions -----

def sigmoid(x):

    return 1/(1+np.exp(-x))

def sigmoid_derivative(x):

    return x*(1-x)

# ----- XOR Dataset -----

X = np.array([[0,0],[0,1],[1,0],[1,1]])

y = np.array([[0],[1],[1],[0]])
```

```

# ----- Hyperparameters -----

learning_rate = 0.1

epochs = 10000

np.random.seed(42)


# ----- Network Initialization (2-2-1) -----

W1 = np.random.uniform(size=(2,2))

b1 = np.random.uniform(size=(1,2))

W2 = np.random.uniform(size=(2,1))

b2 = np.random.uniform(size=(1,1))


losses=[]

accuracies=[]


# ----- Output Header -----

print(f'{'Epoch':<10} | {'Loss':<12} | {'Accuracy (%)':<14} | {'W2 Sample'}')

print("-"*65)


# ----- Training Loop -----

for epoch in range(epochs):


    # Forward Propagation

    hidden = sigmoid(np.dot(X,W1)+b1)

    output = sigmoid(np.dot(hidden,W2)+b2)

```

```

# Metrics

error = y-output

loss = np.mean(np.square(error))

losses.append(loss)


preds = (output>0.5).astype(int)

accuracy = np.mean(preds==y)*100

accuracies.append(accuracy)


# Backpropagation

d_output = error*sigmoid_derivative(output)

d_hidden = d_output.dot(W2.T)*sigmoid_derivative(hidden)


# Update Weights

W2 += hidden.T.dot(d_output)*learning_rate

b2 += np.sum(d_output,axis=0,keepdims=True)*learning_rate

W1 += X.T.dot(d_hidden)*learning_rate

b1 += np.sum(d_hidden,axis=0,keepdims=True)*learning_rate


# Print sample rows with W2 value

if epoch % 1000 == 0:

    print(f"epoch:<10} | {loss:<12.6f} | {accuracy:<14.1f} | {W2.flatten()[0]:.4f}")


print("-"*65)

print("Training Complete")

```

```
# ----- Plot 1: Training Loss -----
```

```
plt.figure(figsize=(8,5))  
plt.plot(losses)  
plt.title("Training Loss (MSE)")  
plt.xlabel("Epochs")  
plt.ylabel("Loss")  
plt.grid(True)  
plt.show()
```

```
# ----- Plot 2: Accuracy Curve -----
```

```
plt.figure(figsize=(8,5))  
plt.plot(accuracies)  
plt.title("Accuracy Curve")  
plt.xlabel("Epochs")  
plt.ylabel("Accuracy (%)")  
plt.grid(True)  
plt.show()
```

```
# ----- Plot 3: Final Decision Boundary -----
```

```
h=0.02  
  
x_min,x_max=X[:,0].min()-0.5,X[:,0].max()+0.5  
y_min,y_max=X[:,1].min()-0.5,X[:,1].max()+0.5  
  
xx,yy=np.meshgrid(np.arange(x_min,x_max,h),  
                  np.arange(y_min,y_max,h))
```

```

grid=np.c_[xx.ravel(),yy.ravel()]

hidden_grid=sigmoid(np.dot(grid,W1)+b1)

out_grid=sigmoid(np.dot(hidden_grid,W2)+b2)

out_grid=out_grid.reshape(xx.shape)


plt.figure(figsize=(8,5))

plt.contourf(xx,yy,out_grid,alpha=0.8)

plt.scatter(X[:,0],X[:,1],c=y.flatten(),edgecolors='k',s=120)

plt.title("Final Decision Boundary")

plt.xlabel("Input 1")

plt.ylabel("Input 2")

plt.show()

```

Sample Output Table

Epoch	Loss	Accuracy (%)	W2 Sample

0	0.324659	50.0	0.0456
1000	0.240589	75.0	-0.5440
2000	0.196030	75.0	-1.3640
3000	0.120663	100.0	-3.2705
4000	0.030459	100.0	-5.6507
5000	0.012541	100.0	-6.6992
6000	0.007368	100.0	-7.2660
7000	0.005093	100.0	-7.6424
8000	0.003847	100.0	-7.9210

9000 | 0.003071 | 100.0 | -8.1409

Training Complete

Result

The neural network successfully learned the XOR logic achieving 100% accuracy and produced a nonlinear decision boundary.





